DISCRETE MATHEMATICS / COURSE PROJECT

ExamCraft

University Exam & Timetable Scheduler

From shared registrations to conflict-free exam slots.

THE CENTRAL IDEA

A course is a vertex. Shared students create edges. Colors are exam slots. A proper coloring gives every student a timetable without simultaneous exams.

PROJECT	DETAIL
Institution	Vishwakarma Institute of Technology, Pune
Programme	B.Tech. Computer Engineering
Student	Prasanna Mahalpure
Academic year	2026-2027
Team / division / guide	Complete the explicit placeholders in docs/REPORT.md before submission.

A practical companion to the complete source code, detailed academic report, reproducible datasets, test suites, and viva guide. Prepared 28 October 2026.

Reading map: Model 02 / Algorithms 03 / Architecture 04 / Data 05 / Worked example 06 / Results 07 / Verification 08 / Demo 09 / Scope & viva 10

PROJECT GUIDE / 02

The mathematical model

Let S be the students, V the distinct course IDs, and R_s the set of courses registered by student s. The conflict graph G = (V, E) is simple and undirected:

EDGE RULE

$\{u, v\} \in E$ exactly when $u \neq v$ and at least one student registers for both u and v. The edge weight w(u,v) counts those shared students.

The graph has no self-loops and no parallel edges. Shared registrations establish a symmetric relation: if A conflicts with B, then B conflicts with A. Repeated registration rows must never inflate weights.

A coloring is an assignment $c: V \to \{0, 1, \ldots, k - 1\}$ satisfying $c(u) \neq c(v)$ for every edge. Internally, colors start at zero; timetable exports display Slot 1 through Slot k. The minimum k is the chromatic number $\chi(G)$.

Why proper coloring means no clashes

Forward direction. Suppose a student has distinct registered courses u and v in the same slot. Their registrations create the edge {u,v}. A proper coloring requires different colors at its endpoints, contradicting the assumed clash.

Reverse direction. If adjacent courses share a slot, the edge certifies that a student takes both. That student would face simultaneous exams. Therefore every clash-free complete assignment is a proper coloring.

Completeness is essential: a missing exam may hide a conflict. Validation must verify that every declared course appears exactly once, as well as checking collisions.

Bounds turn a good result into a certificate

$$L \leq \omega(G) \leq \chi(G) \leq k_found \leq \Delta(G) + 1$$

Here L is the size of an explicitly discovered clique, $\omega(G)$ the maximum clique size, and $\Delta(G)$ the maximum degree. A found clique may be smaller than a maximum clique but is always a valid lower bound. A validated coloring supplies the upper bound k_found.

If a valid schedule uses L slots, both bounds meet and $\chi(G) = L$. A completed exact search can also certify $\chi(G)$, even when the clique certificate is weaker.

PROJECT GUIDE / 03

Three scheduling strategies

01 / Welsh-Powell: fixed degree order

Sort vertices by descending degree and then course ID. Scan that order to build one maximal independent color class. Repeat the scan with a new color until every course is assigned. The implementation follows this color-class formulation explicitly.

```
order = sort(courses, by=(-degree, course_id))
while an uncolored course remains:
    create the next slot
    for course in order:
        if uncolored and compatible with this slot:
            assign course to this slot
```

02 / DSatur: adapt to the partial coloring

Saturation is the number of **distinct colors** among already colored neighbors. At each step, choose highest saturation, then highest total degree, then course ID. Give that course its smallest available color. Four colored neighbors all in one color give saturation one.

03 / Bounded exact search: improve and prove

Start from a valid heuristic incumbent. Branch on legal colors in DSatur order and prune choices unable to improve the incumbent. Introduce new colors canonically to remove redundant color-label permutations. A completed improvement search or matching lower bound proves optimality.

MODE	BUDGET / CLAIM
Browser exact	At most 18 courses; UI budget 1.5 seconds / 100,000 nodes.
Python exact	At most 60 courses; default search budget 3 seconds / 100,000 nodes.
Interrupted search	Return the best valid incumbent; minimum remains unproved.

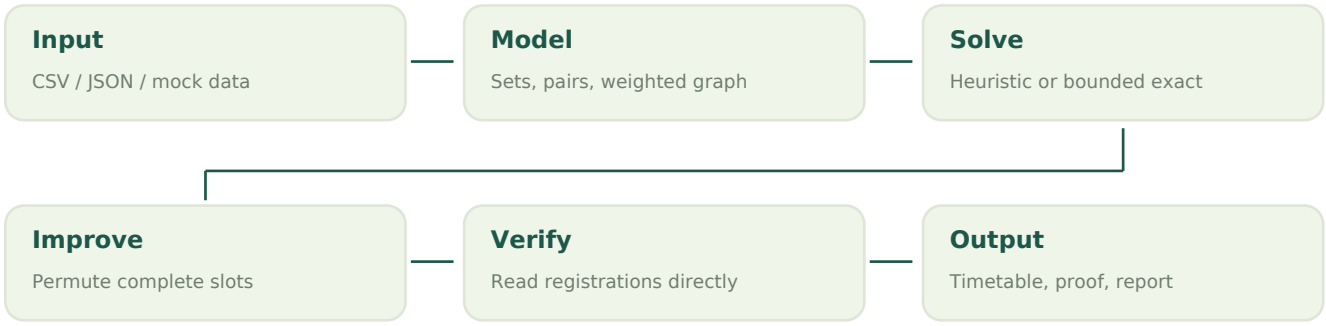
MATHEMATICAL HONESTY

Finding $\chi(G)$ is NP-hard. Deciding whether at most k colors suffice is NP-complete in general and for fixed $k \geq 3$. Verifying a proposed coloring is polynomial. Neither heuristic always finds the minimum or always outperforms the other.

These teaching implementations recompute saturation and use clear set operations. They favor inspectability over sophisticated priority queues. Exact search remains exponential in the worst case.

PROJECT GUIDE / 04

Architecture & constraint flow



Both interfaces implement the same graph-theoretic model. They are independent implementations; the web app is not a client of a hidden Python API.

LAYER	BROWSER	PYTHON
Input	File / pasted CSV or JSON; cohort generator	File parsing; uniform seeded generator
Computation	ES module in a Web Worker	Standard-library scheduler package
Views	Timetable, graph, comparison, student search	CLI tables; exported HTML report
Delivery	CSV, JSON, ICS, HTML, print controls	CSV, schedule / graph / conflict JSON, HTML

Graph construction from registration pairs

Create an adjacency set for each course, including zero-enrollment catalog entries. For every student with r distinct courses, enumerate r(r - 1)/2 unordered pairs. Add both adjacency directions and increment each pair weight once for that student.

Expected set/dictionary work is proportional to $\Sigma_s |R_s|^2$, plus initialization and sorting for deterministic output. Adjacency storage is $O(|V| + |E|)$, excluding the registrations. An adjacency matrix would consume $O(|V|^2)$ cells even on a sparse graph.

A second, independent verification path

After coloring and any slot reordering, read the original students again and group each student’s courses by assigned slot. Reject missing courses, unexpected assignments, invalid slot values, or groups containing more than one registered exam. This avoids merely trusting the graph routine that created the schedule.

Browser datasets stay in memory for the current visit. Reloading restores the demo; download registrations to retain changes. The scheduling application sends no registrations to a scheduling backend.

PROJECT GUIDE / 05

Data contracts & reproducibility

Long-format CSV

```
student_id,course_id,course_name
S001,DM,Discrete Mathematics
S001,DS,Data Structures
S002,DS,Data Structures
S002,OS,Operating Systems
```

One row is one registration; repeated student IDs are expected. course_name is optional. Preserve identifiers as strings, including leading zeroes. CSV supports quoted commas, escaped quotation marks, UTF-8 BOM, and CRLF. Use JSON to represent a zero-enrollment course or a student with no registrations.

Canonical JSON

```
{
  "courses": [{"id": "DM", "name": "Discrete Mathematics"},
              {"id": "DS", "name": "Data Structures"}],
  "students": [
    {"student_id": "S001", "courses": ["DM", "DS"]}
  ]
}
```

For portability, provide a complete unique catalog and distinct courses within each student record. CSV duplicate registrations are deduplicated. Python’s explicit JSON parser rejects duplicate registrations and unknown references in a supplied catalog; the browser normalizes duplicates and may add referenced courses. Both reject duplicate student identities in JSON.

ENTRY POINT	LIMITS
Browser interface	120 courses; 5,000 students; 5 MiB imports
Browser engine	200 courses; 10,000 students; 100,000 registrations
Python input	200 courses; 20,000 students; see DATA_FORMAT.md for file / registration caps

Keep experiments comparable

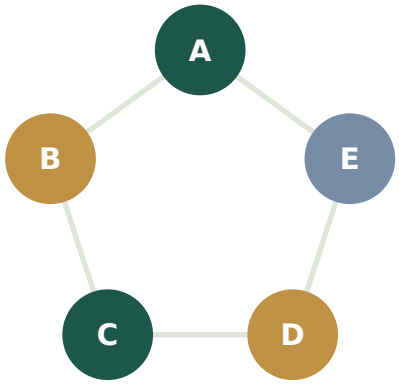
The browser generator favors groups of eight courses with occasional cross-cohort choices. The Python generator samples uniformly and uses a different pseudorandom generator. Equal seeds across languages do not imply equal data. Import examples/cohort_30_300.json into both to compare the same graph.

Use fixed input files, deterministic tie-breakers, matching calendar settings, and recorded budgets. Runtime is machine-dependent; observed timing is evidence for this experiment, not a universal performance guarantee.

PROJECT GUIDE / 06

Worked example: the five-cycle

Five students register for pairs AB, BC, CD, DE, and EA. Each registration pair contributes one edge. The graph is C_5, an odd cycle with five vertices, five edges, and degree two at every vertex.



A valid 3-coloring

Slot 1: A, C

Slot 2: B, D

Slot 3: E

5 vertices / 5 conflict edges

STUDENT	REGISTERED COURSES	CONFLICT CREATED
S1	A, B	A - B
S2	B, C	B - C
S3	C, D	C - D
S4	D, E	D - E
S5	E, A	E - A

Why two slots cannot work

Assume A is in slot 1. Alternating around the cycle forces B into slot 2, C into slot 1, D into slot 2, and E into slot 1. But E is adjacent to A, so that assignment clashes. An odd cycle cannot be properly colored with two colors.

The displayed three-color assignment is valid, proving $\chi(C_5) = 3$. The maximum clique contains only two vertices: there are no triangles. Therefore $\omega(C_5) = 2 < \chi(C_5) = 3$.

A GOOD VIVA MOMENT

The clique lower bound does not always equal the chromatic number. A heuristic may return 3 with bounds [2,3]; the exact solver proves 3 by exhausting the improvement search and tightens the certified interval to [3,3].

Reproduce: python3 -m scheduler solve examples/cycle5.json --algorithm exact --output-dir output/cycle5

PROJECT GUIDE / 07

Results & soft constraints

The shared cohort fixture has **30 courses, 300 students, 1,356 registrations, and 316 conflict edges**. Its density is 316 / 435 = 72.6%. The Python results bundled in examples/results were generated from this fixed file.

PYTHON STRATEGY	SLOTS	CERTIFIED INTERVAL	PROOF
Welsh-Powell	13	$11 \le \chi(G) \le 13$	Unproved
DSatur	13	$11 \le \chi(G) \le 13$	Unproved
Exact search	12	$12 \le \chi(G) \le 12$	Minimum proved

The exact proof completes in 593 search nodes for this fixture. The stored run took about 14 ms, but timings vary by machine and process. The clique certificate still has size 11: the stronger lower bound of 12 comes from completed search, not a falsely enlarged clique.

Gap optimization preserves the hard constraints

For each student, count pairs of exams in adjacent slots on the same day. Three consecutive periods contribute two pairs. A day boundary contributes no pair. This deliberately narrow penalty does not model all forms of student fatigue.

Reorder complete color classes by a one-to-one permutation π. If $c(u) \neq c(v)$, then $\pi(c(u)) \neq \pi(c(v))$. Therefore every edge remains properly colored, all courses stay assigned, and the slot count stays unchanged.

PYTHON RUN / 3 SLOTS PER DAY	BEFORE	AFTER	EXAM DAYS
DSatur / 13 slots	253 pairs	108 pairs	5
Exact / 12 slots	328 pairs	154 pairs	4

FEWER SLOTS CAN MEAN TIGHTER EXAM SPACING

The proved 12-slot schedule uses fewer days, yet its final consecutive-pair count is higher than this 13-slot schedule. Minimum slot count and best student spacing are separate objectives. Report both; do not claim one metric proves overall superiority.

Python uses two sweeps of improving color-class swaps. The browser uses bounded best-improvement swaps, so optimized schedules can differ. Each accepted change lowers the selected penalty; neither method proves a global soft optimum.

PROJECT GUIDE / 08

Verification & evidence

The project suites pass **41 named tests: 18 Python tests and 23 JavaScript tests** (13 engine tests and 10 export tests). Two Python tests use Node when available and are skipped when it is absent. Node is a testing tool; the browser app and Python CLI do not require npm dependencies.

EVIDENCE	WHAT IT CHECKS
Known graph families	Cliques, cycles, bipartite and isolated-vertex cases have expected proper colorings.
Independent exact oracle	Python checks 105 small random graphs. JavaScript checks all 1,024 five-vertex graphs and 70 random graphs with 6-9 vertices.
Budget exhaustion	Time or node cutoffs retain a valid incumbent and do not falsely mark the result optimal.
Import validation	Quoted CSV, duplicates, malformed input, ambiguous identities, special object-key identifiers.
Cross-engine checks	Shared graph adjacency / weights, deterministic heuristic assignments before reordering, and browser exports validated by Python.
Soft and calendar checks	Nonincreasing penalty, preserved coverage and slot count, weekend advancement, partial exam days.
CLI integration	End-to-end export, independently validated assignments, and deliberate tamper detection.

Re-run the checks

```
python3 -m unittest discover -s tests -v
node --test tests/test_web_engine.mjs tests/test_exporters.mjs
```

Use Python 3.10+ and Node 18+ for the complete suites. Detailed current browser observations, export checks, and verification limits belong in docs/VALIDATION.md. Browser checks include graph inspection, student lookup, valid / invalid CSV paste, C5 exact proof, and weekend / duration settings.

Browser workflow and export checks

The actual browser JSON download was independently validated in Python against all 300 students and 1,356 registrations. Export tests check CSV safety, ICS Unicode folding, and dates. Repeated download notifications were limited in the test browser; complete cross-browser download behavior remains unverified.

No unit-test collection proves every possible graph correct. Combining independent brute-force comparisons, known families, invariants, cross-implementation checks, and live UI verification gives stronger evidence than one attractive screenshot.

PROJECT GUIDE / 09

Run it & present it

Offline browser app: one command

```
python3 run.py
```

Run from the extracted project root. The launcher serves only dist/ on the loopback interface, opens a browser, and prints the local URL. Press Ctrl+C to stop. Use --port 8001 if port 8000 is busy, or --no-browser to open the address manually. On Windows, use py in place of python3 if appropriate.

CLI demonstration on a fixed dataset

```
python3 -m scheduler compare examples/cohort_30_300.json --exact

python3 -m scheduler solve examples/cohort_30_300.json \
  --algorithm dsatur --output-dir output/demo \
  --slots-per-day 3 --skip-weekends --optimize-gaps

python3 -m scheduler validate examples/cohort_30_300.json \
  output/demo/schedule.json
```

TIME	DEMONSTRATION	POINT TO EXPLAIN
0:00-1:00	State the problem and mapping	Course = vertex; conflict = edge; slot = color.
1:00-2:00	Inspect a conflict graph node	An edge exists even for one shared student.
2:00-3:00	Compare heuristics and bounds	A proper 13-slot result need not be minimum.
3:00-4:00	Use C5 and exact search	Clique bound 2, but chromatic number 3.
4:00-5:00	Find a student and export	Independent validation checks the real hard constraint.
5:00-6:00	Show cohort exact result	12 slots are proved; soft spacing has a trade-off.

Before presenting: keep the terminal open, test on the presentation laptop, extract the entire archive, and have the bundled HTML results available as a fallback. Avoid relying on an internet connection for the classroom demonstration.

The detailed narrative, optional role split, and contingency steps are in docs/DEMO_SCRIPT.md. Replace team, batch, and instructor placeholders in the academic report before submitting.

PROJECT GUIDE / 10

Scope, viva & references

What this version guarantees

For the supplied registrations, a validated proper coloring assigns every course one slot and gives each student no simultaneous exams. The app clearly reports whether minimum slot count was proved. It can improve a defined consecutive-pair penalty while retaining feasibility.

What needs an expanded model

Room capacity, room allocation, invigilators, holidays, travel time, individual accommodations, and different per-course exam durations are outside the implemented constraints. The browser supports a common period duration for presentation and calendar exports. Operational university deployment would also need institutional data controls and review.

VIVA PROMPT	ANSWER TO RETAIN
Why is the graph undirected?	Shared enrollment is symmetric.
Degree vs saturation?	Degree counts neighbors; saturation counts distinct colors of colored neighbors.
Why validate coverage?	An omitted exam can hide a clash.
Why does a clique bound work?	Pairwise conflicting courses all need different slots.
Can a heuristic be proved optimal?	Yes, when a valid lower bound meets its slot count.
Why do implementations differ?	Different generators, budgets, and gap searches; compare shared input and guarantees.

Original algorithm and complexity references

1. D. J. A. Welsh and M. B. Powell (1967). An upper bound for the chromatic number of a graph and its application to timetabling problems. The Computer Journal 10(1), 85-86. DOI: 10.1093/comjnl/10.1.85.

2. Daniel Brélaz (1979). New methods to color the vertices of a graph. Communications of the ACM 22(4), 251-256. DOI: 10.1145/359094.359101.

3. Richard M. Karp (1972). Reducibility Among Combinatorial Problems. Complexity of Computer Computations, 85-103. DOI: 10.1007/978-1-4684-2001-2_9.

Source of project-specific results: included inputs, examples/results/*/schedule.json, and the independent test suites. Full discussion, proofs, complexity notes, and source links: docs/REPORT.md. Full viva set: docs/VIVA.md.